summary_analysis

```
library(arrow)
```

```
Attaching package: 'arrow'

The following object is masked from 'package:utils':

    timestamp
```

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(readr)
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2

-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x lubridate::duration() masks arrow::duration()
x dplyr::filter()       masks stats::filter()
x dplyr::lag()          masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

```r
library(moments)

# Load the two CSV files
rounds <- read_csv("data/analysis/rounds.csv", show_col_types = FALSE)
round_states <- read_csv("data/analysis/round_states.csv", show_col_types = FALSE)

# Join keys: your files use round_num, not round_id
join_keys <- c("match_id", "round_num")

# Keep only columns from rounds that aren't already in round_states
round_info <- rounds %>%
  select(
    all_of(join_keys),
    is_overtime,
    is_pistol_round,
    ct_won_previous_round,
    ct_buy_type,
    t_buy_type,
    ct_equipment_freeze_end,
    t_equipment_freeze_end,
    first_death_side,
    freeze_end_tick,
    plant_tick,
    end_tick,
    round_duration_s,
    winner_team,
    reason_name
  )

# Attach round-level information to every snapshot
```

```r
round_states_enriched <- round_states %>%
  left_join(round_info, by = join_keys)

# Verify the result
cat("Original snapshot rows:", nrow(round_states), "\n")
```

```
Original snapshot rows: 116201
```

```r
cat("Merged snapshot rows:", nrow(round_states_enriched), "\n")
```

```
Merged snapshot rows: 116201
```

```r
cat("Merged columns:", ncol(round_states_enriched), "\n")
```

```
Merged columns: 59
```

```r
# Save the merged dataset
write_csv(round_states_enriched, "data/analysis/round_states_enriched.csv")

write_parquet(
  round_states_enriched,
  "data/analysis/round_states_enriched.parquet"
)
```

Quick view of merged file

```r
round_states <- read_parquet("data/analysis/round_states_enriched.parquet")

# View the data (only in an interactive session; fails when rendering)
if (interactive()) View(round_states)

# Inspect column names and data types
str(round_states)
```

```
spc_tbl_ [116,201 x 59] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ match_id               : chr [1:116201] "9z-vs-g2-m1-dust2" "9z-vs-g2-m1-dust2" "9z-vs-g2
 $ map                    : chr [1:116201] "de_dust2" "de_dust2" "de_dust2" "de_dust2" ...
```

```
$ part                  : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ round_num             : num [1:116201] 1 1 1 1 1 1 1 1 1 1 ...
$ tick                  : num [1:116201] 1684 1748 1812 1876 1940 ...
$ seconds               : num [1:116201] 0 1 2 3 4 5 6 7 8 9 ...
$ time_remaining_s      : num [1:116201] 115 114 113 112 111 ...
$ bomb_planted          : logi [1:116201] FALSE FALSE FALSE FALSE FALSE FALSE ...
$ seconds_since_plant   : num [1:116201] NA NA NA NA NA NA NA NA NA NA ...
$ plant_site            : chr [1:116201] NA NA NA NA ...
$ bomb_carried          : logi [1:116201] FALSE FALSE TRUE TRUE TRUE TRUE ...
$ ct_team               : chr [1:116201] "G2" "G2" "G2" "G2" ...
$ t_team                : chr [1:116201] "9z" "9z" "9z" "9z" ...
$ ct_score_before       : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ t_score_before        : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ ct_alive              : num [1:116201] 5 5 5 5 5 5 5 5 5 5 ...
$ ct_hp                 : num [1:116201] 500 500 500 500 500 500 500 499 498 486 ...
$ ct_armor              : num [1:116201] 300 400 400 400 400 400 400 400 400 393 ...
$ ct_helmets            : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ ct_defusers           : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ ct_equipment          : num [1:116201] 3450 4100 4100 4100 4100 4100 4100 4100 4100 4100
$ ct_money              : num [1:116201] 1350 700 700 700 700 700 700 700 700 700 ...
$ ct_rifles             : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ ct_awps               : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ ct_smgs               : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ ct_smokes             : num [1:116201] 1 1 1 0 0 0 0 0 0 0 ...
$ ct_flashes            : num [1:116201] 1 1 1 1 1 1 1 1 1 1 ...
$ ct_he                 : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ ct_molotovs           : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ t_alive               : num [1:116201] 5 5 5 5 5 5 5 5 5 5 ...
$ t_hp                  : num [1:116201] 500 500 500 500 500 500 500 500 500 500 ...
$ t_armor               : num [1:116201] 300 300 300 300 300 300 300 300 300 300 ...
$ t_helmets             : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ t_defusers            : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ t_equipment           : num [1:116201] 4800 4800 4800 4800 4800 4800 4800 4800 4800 4800
$ t_money               : num [1:116201] 600 600 600 600 600 600 600 600 600 600 ...
$ t_rifles              : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ t_awps                : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ t_smgs                : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ t_smokes              : num [1:116201] 2 2 2 2 2 2 1 1 1 1 ...
$ t_flashes             : num [1:116201] 1 1 1 1 1 1 1 1 1 1 ...
$ t_he                  : num [1:116201] 0 0 0 0 0 0 0 0 0 0 ...
$ t_molotovs            : num [1:116201] 1 1 1 1 1 1 1 1 1 1 ...
$ winner_side           : chr [1:116201] "terrorist" "terrorist" "terrorist" "terrorist" .
$ ct_win                : logi [1:116201] FALSE FALSE FALSE FALSE FALSE FALSE ...
```

```
$ is_overtime            : logi [1:116201] FALSE FALSE FALSE FALSE FALSE FALSE ...
$ is_pistol_round        : logi [1:116201] TRUE TRUE TRUE TRUE TRUE TRUE ...
$ ct_won_previous_round  : logi [1:116201] NA NA NA NA NA NA ...
$ ct_buy_type            : chr [1:116201] "pistol" "pistol" "pistol" "pistol" ...
$ t_buy_type             : chr [1:116201] "pistol" "pistol" "pistol" "pistol" ...
$ ct_equipment_freeze_end: num [1:116201] 3450 3450 3450 3450 3450 3450 3450 3450 3450 3450
$ t_equipment_freeze_end : num [1:116201] 4800 4800 4800 4800 4800 4800 4800 4800 4800 4800
$ first_death_side       : chr [1:116201] "counter-terrorist" "counter-terrorist" "counter-t
$ freeze_end_tick        : num [1:116201] 1684 1684 1684 1684 1684 ...
$ plant_tick             : num [1:116201] 6180 6180 6180 6180 6180 6180 6180 6180 6180 6180
$ end_tick               : num [1:116201] 6069 6069 6069 6069 6069 ...
$ round_duration_s       : num [1:116201] 68.5 68.5 68.5 68.5 68.5 ...
$ winner_team            : chr [1:116201] "9z" "9z" "9z" "9z" ...
$ reason_name            : chr [1:116201] "terrorists_win" "terrorists_win" "terrorists_win"
- attr(*, "spec")=
 .. cols(
 ..   match_id = col_character(),
 ..   map = col_character(),
 ..   part = col_double(),
 ..   round_num = col_double(),
 ..   tick = col_double(),
 ..   seconds = col_double(),
 ..   time_remaining_s = col_double(),
 ..   bomb_planted = col_logical(),
 ..   seconds_since_plant = col_double(),
 ..   plant_site = col_character(),
 ..   bomb_carried = col_logical(),
 ..   ct_team = col_character(),
 ..   t_team = col_character(),
 ..   ct_score_before = col_double(),
 ..   t_score_before = col_double(),
 ..   ct_alive = col_double(),
 ..   ct_hp = col_double(),
 ..   ct_armor = col_double(),
 ..   ct_helmets = col_double(),
 ..   ct_defusers = col_double(),
 ..   ct_equipment = col_double(),
 ..   ct_money = col_double(),
 ..   ct_rifles = col_double(),
 ..   ct_awps = col_double(),
 ..   ct_smgs = col_double(),
 ..   ct_smokes = col_double(),
 ..   ct_flashes = col_double(),
```

```
..   ct_he = col_double(),
..   ct_molotovs = col_double(),
..   t_alive = col_double(),
..   t_hp = col_double(),
..   t_armor = col_double(),
..   t_helmets = col_double(),
..   t_defusers = col_double(),
..   t_equipment = col_double(),
..   t_money = col_double(),
..   t_rifles = col_double(),
..   t_awps = col_double(),
..   t_smgs = col_double(),
..   t_smokes = col_double(),
..   t_flashes = col_double(),
..   t_he = col_double(),
..   t_molotovs = col_double(),
..   winner_side = col_character(),
..   ct_win = col_logical()
.. )
```

```
# First few rows
head(round_states)
```

```
# A tibble: 6 x 59
  match_id     map    part round_num  tick seconds time_remaining_s bomb_planted
  <chr>        <chr> <dbl>     <dbl> <dbl>   <dbl>            <dbl> <lgl>
1 9z-vs-g2-m1~ de_d~     0         1  1684       0             115. FALSE
2 9z-vs-g2-m1~ de_d~     0         1  1748       1             114. FALSE
3 9z-vs-g2-m1~ de_d~     0         1  1812       2             113. FALSE
4 9z-vs-g2-m1~ de_d~     0         1  1876       3             112. FALSE
5 9z-vs-g2-m1~ de_d~     0         1  1940       4             111. FALSE
6 9z-vs-g2-m1~ de_d~     0         1  2004       5             110. FALSE
# i 51 more variables: seconds_since_plant <dbl>, plant_site <chr>,
#   bomb_carried <lgl>, ct_team <chr>, t_team <chr>, ct_score_before <dbl>,
#   t_score_before <dbl>, ct_alive <dbl>, ct_hp <dbl>, ct_armor <dbl>,
#   ct_helmets <dbl>, ct_defusers <dbl>, ct_equipment <dbl>, ct_money <dbl>,
#   ct_rifles <dbl>, ct_awps <dbl>, ct_smgs <dbl>, ct_smokes <dbl>,
#   ct_flashes <dbl>, ct_he <dbl>, ct_molotovs <dbl>, t_alive <dbl>,
#   t_hp <dbl>, t_armor <dbl>, t_helmets <dbl>, t_defusers <dbl>, ...
```

```r
# Number of rows and columns
dim(round_states)
```

```
[1] 116201     59
```

```r
unique_round_count <- round_states %>%
  group_by(match_id) %>%
  summarise(unique_rounds = n_distinct(round_num)) %>%
  ungroup() %>%
  collect()

unique_round_count |> summary()
```

```
      match_id  unique_rounds
 Length   :60   Min.   :14.00
 N.unique :60   1st Qu.:18.75
 N.blank  : 0   Median :21.00
 Min.nchar:17   Mean   :20.95
 Max.nchar:35   3rd Qu.:22.25
                Max.   :35.00
```

```r
unique_round_count %>%
  summarise(skew = skewness(unique_rounds))
```

```
# A tibble: 1 x 1
   skew
  <dbl>
1  1.13
```

unique round filter (GLOBAL)

```r
unique_round_filter <- round_states %>%
  distinct(match_id, round_num, map) %>%
  collect()
```

```r
filtered_rounds_global <- round_states %>%
  semi_join(
    unique_round_filter,
    by = c("match_id", "round_num")
  )
```

unique round filter (BY MAP)

```
unique_round_filter_maps <- round_states %>%
  distinct(match_id, round_num, map) %>%
  collect()
```

```
filtered_rounds_map <- round_states %>%
  semi_join(
    unique_round_filter,
    by = c("match_id", "round_num", "map")
  )
```

Round-end summaries

For each round, use the latest recorded snapshot at or before `end_tick`. Because snapshots are sampled periodically, this is an approximation of the round-end state; an exact `tick == end_tick` row may not exist.

check

```
filtered_rounds_global %>%
  filter(tick <= end_tick) %>%
  group_by(match_id, round_num) %>%
  slice_max(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  summarise(
    mean_t_health_global = mean(t_hp, na.rm = TRUE),
    mean_ct_health_global = mean(ct_hp, na.rm = TRUE),
    sd_t_health_global = sd(t_hp, na.rm = TRUE),
    sd_ct_health_global = sd(ct_hp, na.rm = TRUE)
  ) %>%
  collect()
```

```
# A tibble: 1 x 4
  mean_t_health_global mean_ct_health_global sd_t_health_global
                 <dbl>                 <dbl>              <dbl>
1                 135.                  190.               126.
# i 1 more variable: sd_ct_health_global <dbl>
```

```r
filtered_rounds_map %>%
  filter(tick <= end_tick) %>%
  group_by(match_id, round_num, map) %>%
  slice_max(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  group_by(map) %>%
  summarise(
    ## Means
    mean_t_alive_map = mean(t_alive, na.rm = TRUE),
    mean_t_health_map = mean(t_hp, na.rm = TRUE),
    mean_ct_alive_map = mean(ct_alive, na.rm = TRUE),
    mean_ct_health_map = mean(ct_hp, na.rm = TRUE),
    ## SD
    sd_t_alive_map = sd(t_alive, na.rm = TRUE),
    sd_t_health_map = sd(t_hp, na.rm = TRUE),
    sd_ct_alive_map = sd(ct_alive, na.rm = TRUE),
    sd_ct_health_map = sd(ct_hp, na.rm = TRUE)
  ) %>%
  collect()
```

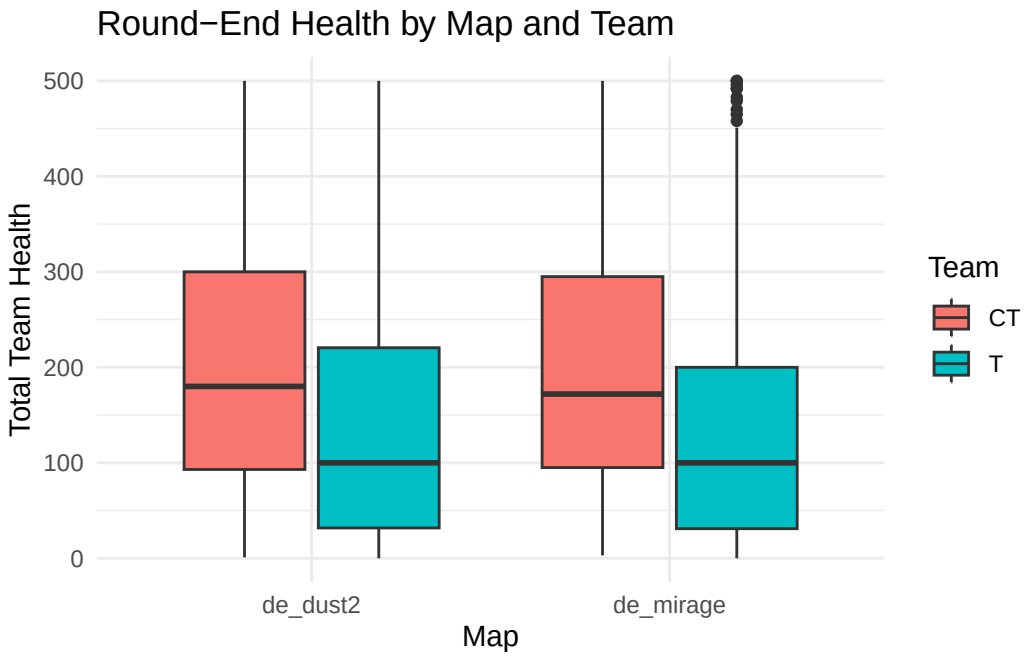
```
# A tibble: 2 x 9
  map    mean_t_alive_map mean_t_health_map mean_ct_alive_map mean_ct_health_map
  <chr>             <dbl>             <dbl>             <dbl>              <dbl>
1 de_du~             1.91              142.              2.33               191.
2 de_mi~             1.76              128.              2.35               190.
# i 4 more variables: sd_t_alive_map <dbl>, sd_t_health_map <dbl>,
#   sd_ct_alive_map <dbl>, sd_ct_health_map <dbl>
```

```r
round_end_health <- filtered_rounds_map %>%
  filter(tick <= end_tick) %>%
  group_by(match_id, round_num, map) %>%
  slice_max(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  collect()

round_end_health_long <- round_end_health %>%
  pivot_longer(
    cols = c(t_hp, ct_hp),
    names_to = "team",
    values_to = "health"
  ) %>%
```

```
  mutate(
    team = recode(
      team,
      t_hp = "T",
      ct_hp = "CT"
    )
  )

ggplot(round_end_health_long, aes(x = map, y = health, fill = team)) +
  geom_boxplot() +
  labs(
    title = "Round-End Health by Map and Team",
    x = "Map",
    y = "Total Team Health",
    fill = "Team"
  ) +
  theme_minimal()
```

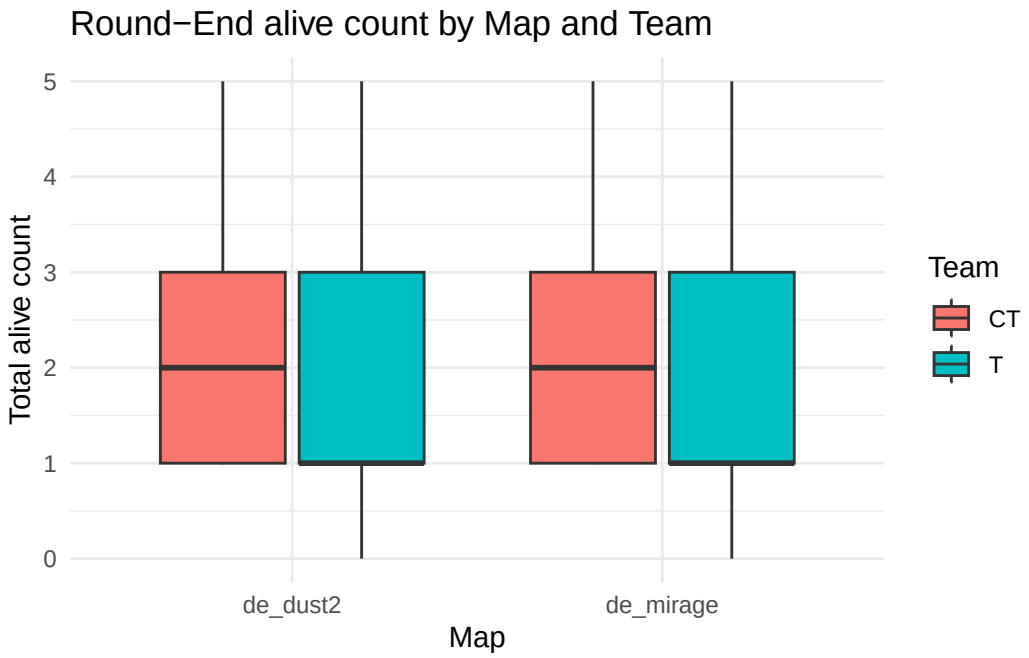



```
round_end_alive <- filtered_rounds_map %>%
  group_by(match_id, round_num, map) %>%
  filter(tick <= end_tick) %>%
  slice_max(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
```

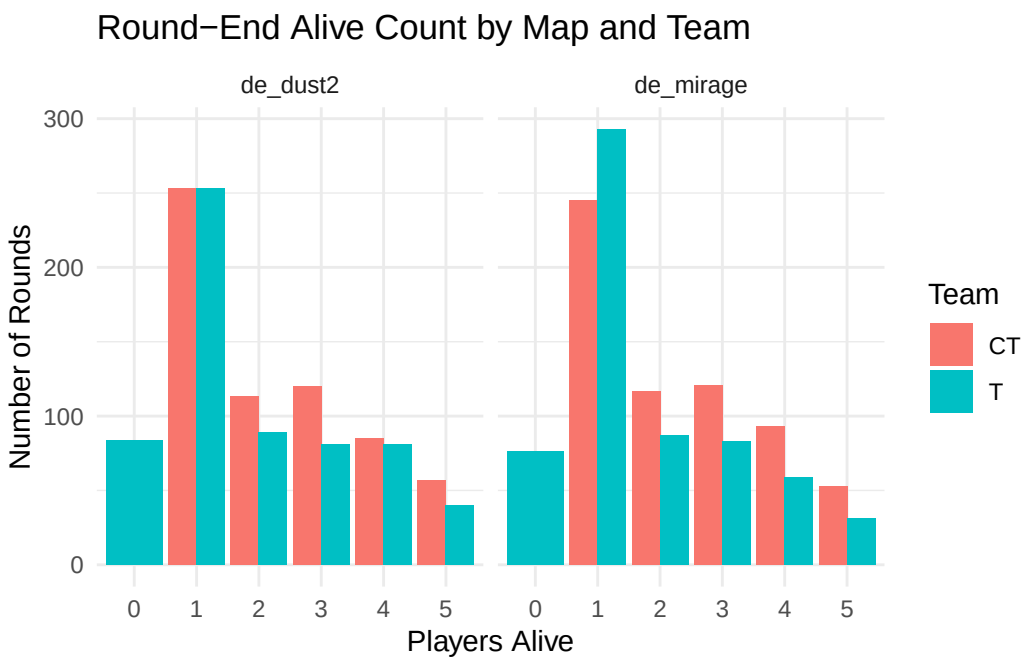
```r
  collect()

round_end_alive_long <- round_end_alive %>%
  pivot_longer(
    cols = c(t_alive, ct_alive),
    names_to = "team",
    values_to = "count"
  ) %>%
  mutate(
    team = recode(
      team,
      t_alive = "T",
      ct_alive = "CT"
    )
  )

ggplot(round_end_alive_long, aes(x = map, y = count, fill = team)) +
  geom_boxplot() +
  labs(
    title = "Round-End alive count by Map and Team",
    x = "Map",
    y = "Total alive count",
    fill = "Team"
  ) +
  theme_minimal()
```




```
round_end_alive_long %>%
  count(map, team, count, name = "rounds") %>%
  ggplot(aes(x = factor(count, levels = 0:5),
             y = rounds, fill = team)) +
  geom_col(position = "dodge") +
  facet_wrap(~map) +
  labs(
    title = "Round-End Alive Count by Map and Team",
    x = "Players Alive",
    y = "Number of Rounds",
    fill = "Team"
  ) +
  theme_minimal()
```



diffs

```
round_end_diffs <- filtered_rounds_map %>%
  filter(tick <= end_tick) %>%
  group_by(match_id, round_num, map) %>%
  slice_max(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  mutate(
    alive_diff = t_alive - ct_alive,
    hp_diff = t_hp - ct_hp
  ) %>%
  collect()

round_end_diffs %>%
  group_by(map) %>%
  summarise(
    mean_alive_diff = mean(alive_diff, na.rm = TRUE),
    sd_alive_diff = sd(alive_diff, na.rm = TRUE),
    mean_hp_diff = mean(hp_diff, na.rm = TRUE),
    sd_hp_diff = sd(hp_diff, na.rm = TRUE),
    rounds = n(),
```

```
    .groups = "drop"
  )
```

```
# A tibble: 2 x 6
  map       mean_alive_diff sd_alive_diff mean_hp_diff sd_hp_diff rounds
  <chr>               <dbl>         <dbl>        <dbl>      <dbl>  <int>
1 de_dust2           -0.424          2.28        -48.9       202.    628
2 de_mirage          -0.591          2.21        -61.5       196.    629
```

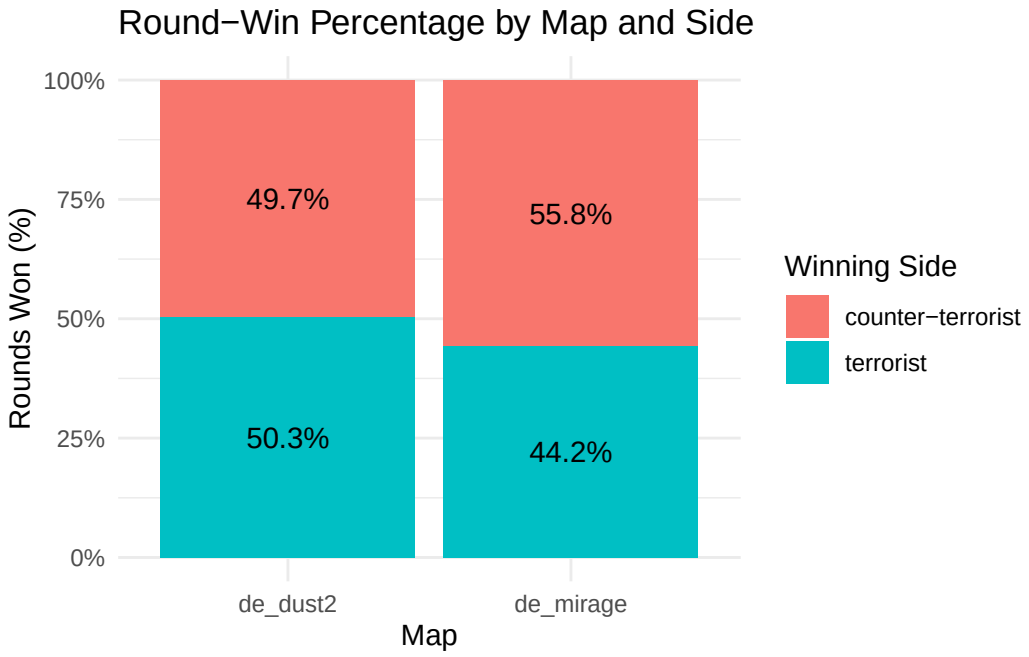
Win Distributions

using unique Rounds

```
win_distributions_map <- filtered_rounds_map %>%
  group_by(match_id, round_num, map) %>%
  filter(tick <= end_tick) %>%
  slice_max(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  distinct(match_id, round_num, map, winner_side) %>%
  group_by(map, winner_side) %>%
  summarise(
    rounds_won = n(),
    .groups = "drop"
  ) %>%
  group_by(map) %>%
  mutate(
    win_percentage = 100 * rounds_won / sum(rounds_won)
  ) %>%
  ungroup() %>%
  collect()

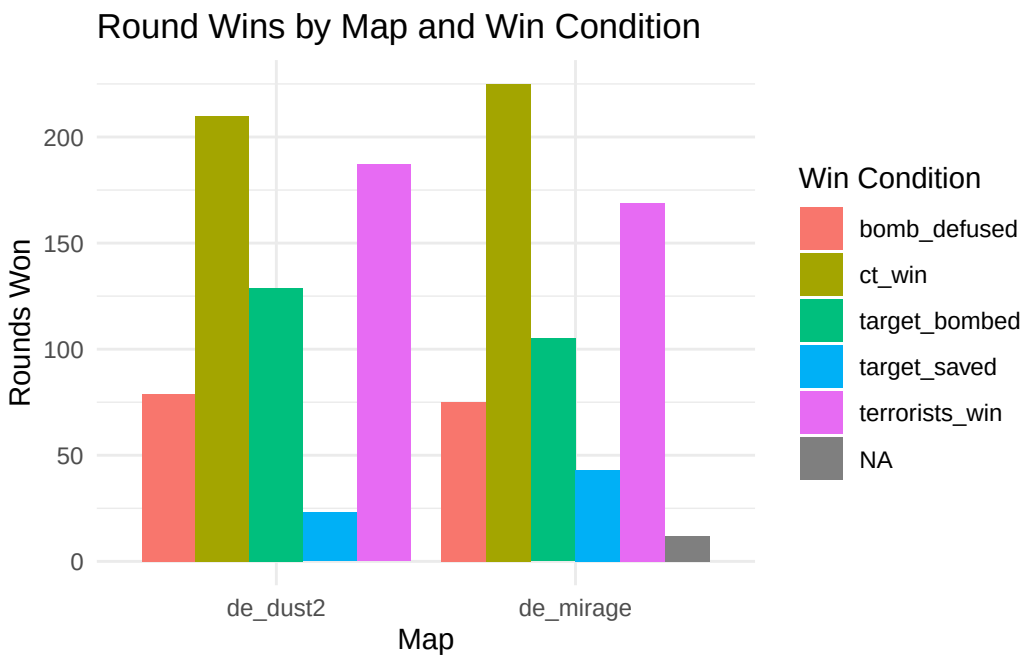
ggplot(win_distributions_map,
       aes(x = map, y = win_percentage, fill = winner_side)) +
  geom_col(position = "stack") +
  geom_text(
    aes(label = sprintf("%.1f%%", win_percentage)),
    position = position_stack(vjust = 0.5)
  ) +
  scale_y_continuous(
    labels = function(x) paste0(x, "%"),
```

```
    limits = c(0, 100)
  ) +
  labs(
    title = "Round-Win Percentage by Map and Side",
    x = "Map",
    y = "Rounds Won (%)",
    fill = "Winning Side"
  ) +
  theme_minimal()
```




```
win_distributions_win_con_map <- filtered_rounds_map %>%
  group_by(match_id, round_num, map) %>%
  filter(tick <= end_tick) %>%
  slice_max(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  distinct(match_id, round_num, map, reason_name) %>%
  group_by(map, reason_name) %>%
  summarise(
    rounds_won = n(),
    .groups = "drop"
  ) %>%
  collect()
```

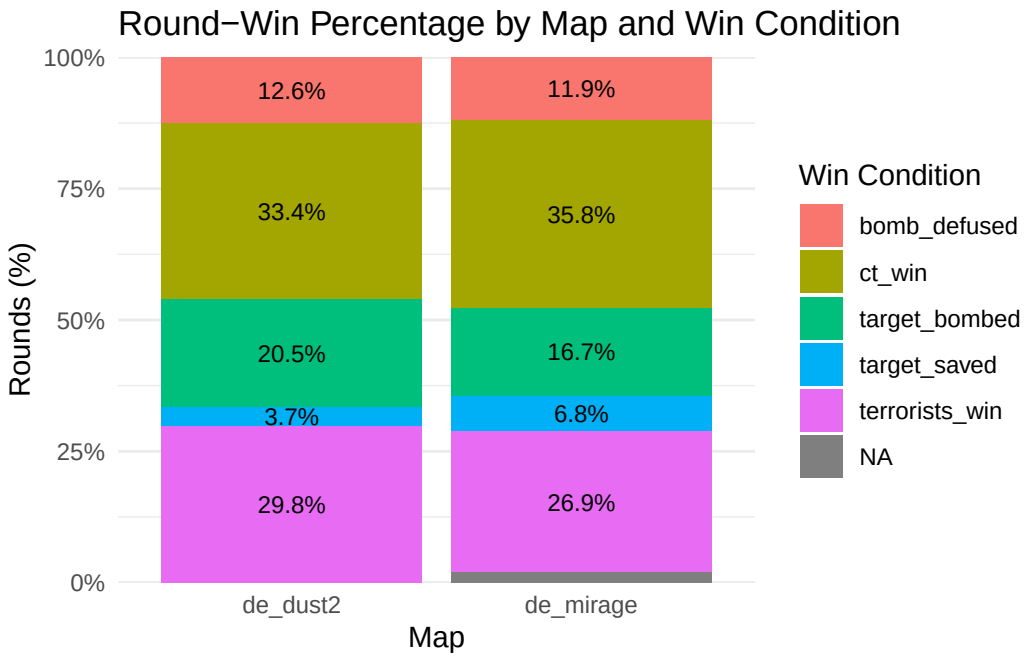
```r
ggplot(win_distributions_win_con_map,
       aes(x = map, y = rounds_won, fill = reason_name)) +
  geom_col(position = "dodge") +
  labs(
    title = "Round Wins by Map and Win Condition",
    x = "Map",
    y = "Rounds Won",
    fill = "Win Condition"
  ) +
  theme_minimal()
```




```r
win_distributions_win_con_map <- filtered_rounds_map %>%
  filter(tick <= end_tick) %>%
  group_by(match_id, round_num, map) %>%
  slice_max(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  distinct(match_id, round_num, map, .keep_all = TRUE) %>%
  group_by(map, reason_name) %>%
  summarise(
    rounds_won = n(),
    .groups = "drop"
  ) %>%
  group_by(map) %>%
```

```
  mutate(
    win_percentage = rounds_won / sum(rounds_won) * 100
  ) %>%
  ungroup() %>%
  collect()

ggplot(
  win_distributions_win_con_map,
  aes(x = map, y = win_percentage, fill = reason_name)
) +
  geom_col() +
  geom_text(
    aes(
      label = if_else(
        win_percentage >= 3,
        sprintf("%.1f%%", win_percentage),
        ""
      )
    ),
    position = position_stack(vjust = 0.5),
    size = 3
  ) +
  scale_y_continuous(
    labels = \(x) paste0(x, "%"),
    limits = c(0, 100),
    expand = c(0, 0)
  ) +
  labs(
    title = "Round-Win Percentage by Map and Win Condition",
    x = "Map",
    y = "Rounds (%)",
    fill = "Win Condition"
  ) +
  theme_minimal()
```



Bomb statistics

```
unique_rounds_bomb <- filtered_rounds_map %>%
  group_by(match_id, round_num, map) %>%
  filter(tick <= end_tick) %>%
  slice_max(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  distinct(match_id, round_num, map, .keep_all = TRUE) %>%
  collect()
##

bomb_plant_summary <- unique_rounds_bomb %>%
  mutate(
    bomb_status = case_when(
      bomb_planted == TRUE  ~ "Planted",
      bomb_planted == FALSE ~ "Not planted",
      TRUE                  ~ "Unknown"
    )
  ) %>%
  count(bomb_status, name = "rounds") %>%
  mutate(
    percentage = 100 * rounds / sum(rounds)
```

```
  )

bomb_plant_summary
```

```
# A tibble: 2 x 3
  bomb_status rounds percentage
  <chr>        <int>      <dbl>
1 Not planted    631       50.2
2 Planted        626       49.8
```

```
planted_bomb_outcomes <- round_states_enriched %>%
  filter(bomb_planted == TRUE) %>%
  group_by(match_id, round_num, map) %>%
  slice_min(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  distinct(match_id, round_num, map, .keep_all = TRUE) %>%
  mutate(
    outcome = case_when(
      reason_name == "bomb_defused" ~ "Bomb defused",
      reason_name == "target_bombed" ~ "Bomb exploded",
      is.na(reason_name) ~ "Unknown outcome",
      TRUE ~ as.character(reason_name)
    )
  ) %>%
  group_by(outcome) %>%
  summarise(
    rounds = n(),

    # Time from round start until bomb plant
    mean_seconds_until_plant =
      mean(seconds, na.rm = TRUE),
    median_seconds_until_plant =
      median(seconds, na.rm = TRUE),
    sd_seconds_until_plant =
      sd(seconds, na.rm = TRUE),

    # Time elapsed after bomb plant
    mean_seconds_after_plant =
      mean(seconds_since_plant, na.rm = TRUE),
    median_seconds_after_plant =
      median(seconds_since_plant, na.rm = TRUE),
```

```
    sd_seconds_after_plant =
      sd(seconds_since_plant, na.rm = TRUE),

    .groups = "drop"
  ) %>%
  collect()

planted_bomb_outcomes
```

```
# A tibble: 4 x 8
  outcome         rounds mean_seconds_until_plant median_seconds_until_plant
  <chr>            <int>                    <dbl>                       <dbl>
1 Bomb defused       154                     77.1                          83
2 Bomb exploded      234                     76.2                          80
3 Unknown outcome      7                     84.1                          91
4 terrorists_win     231                     76.5                          79
# i 4 more variables: sd_seconds_until_plant <dbl>,
#   mean_seconds_after_plant <dbl>, median_seconds_after_plant <dbl>,
#   sd_seconds_after_plant <dbl>
```

Check

```
planted_bomb_outcomes_seconds_since_plant <- round_states_enriched %>%
  filter(bomb_planted == TRUE, tick <= end_tick) %>%
  group_by(match_id, round_num, map) %>%
  slice_max(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  distinct(match_id, round_num, map, .keep_all = TRUE) %>%
  mutate(
    outcome = case_when(
      reason_name == "bomb_defused" ~ "Bomb defused",
      reason_name == "target_bombed" ~ "Bomb exploded",
      is.na(reason_name) ~ "Unknown outcome",
      TRUE ~ as.character(reason_name)
    )
  ) %>%
  group_by(outcome) %>%
  summarise(
    rounds = n(),
    mean_seconds_since_plant =
```

```
      mean(seconds_since_plant, na.rm = TRUE),
    median_seconds_since_plant =
      median(seconds_since_plant, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  collect()

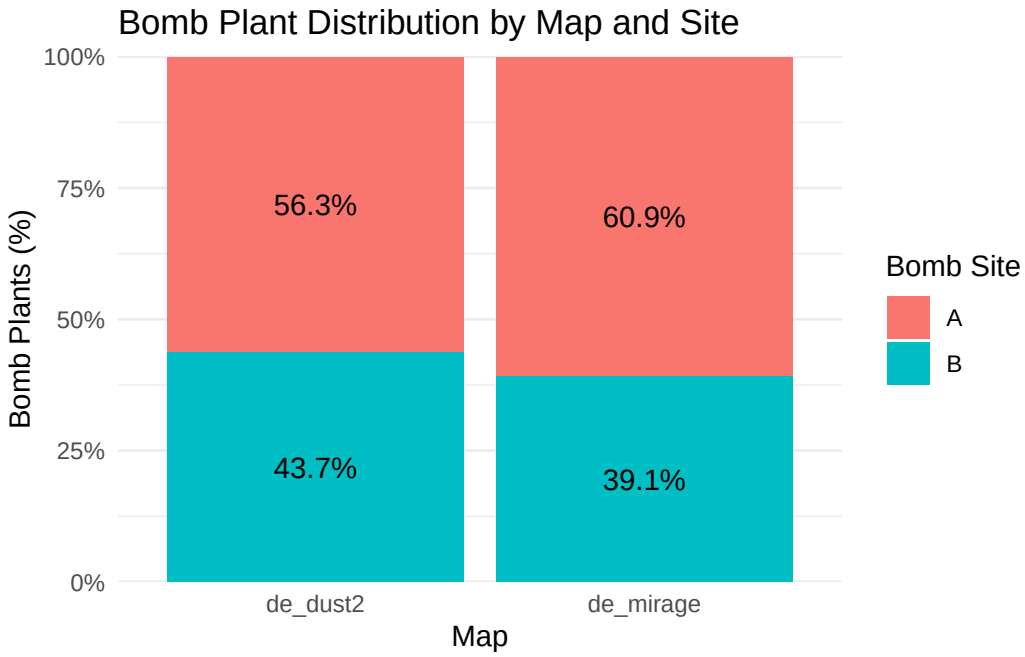
planted_bomb_outcomes_seconds_since_plant
```

```
# A tibble: 4 x 4
  outcome         rounds mean_seconds_since_plant median_seconds_since_plant
  <chr>            <int>                    <dbl>                       <dbl>
1 Bomb defused       154                     26.6                        28.4
2 Bomb exploded      234                     40.5                        40.5
3 Unknown outcome      7                     34.7                        33.9
4 terrorists_win     231                     18.2                        19.0
```

```
bomb_plant_site_map <- round_states_enriched %>%
  filter(bomb_planted == TRUE) %>%
  group_by(match_id, round_num, map) %>%
  slice_min(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  distinct(match_id, round_num, map, .keep_all = TRUE) %>%
  group_by(map, plant_site) %>%
  summarise(
    plants = n(),
    .groups = "drop"
  ) %>%
  group_by(map) %>%
  mutate(
    percentage = 100 * plants / sum(plants)
  ) %>%
  ungroup() %>%
  collect()

ggplot(
  bomb_plant_site_map,
  aes(x = map, y = percentage, fill = plant_site)
) +
  geom_col(position = "stack") +
  geom_text(
```

```
    aes(label = sprintf("%.1f%%", percentage)),
    position = position_stack(vjust = 0.5)
  ) +
  scale_y_continuous(
    labels = \(x) paste0(x, "%"),
    limits = c(0, 100),
    expand = c(0, 0)
  ) +
  labs(
    title = "Bomb Plant Distribution by Map and Site",
    x = "Map",
    y = "Bomb Plants (%)",
    fill = "Bomb Site"
  ) +
  theme_minimal()
```




```
library(tidyverse)

plant_timing <- round_states_enriched %>%
  filter(bomb_planted == TRUE) %>%
  group_by(match_id, round_num, map) %>%
  slice_min(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
```

```
  select(match_id, round_num, map, seconds, reason_name)

post_plant_timing <- round_states_enriched %>%
  filter(bomb_planted == TRUE, tick <= end_tick) %>%
  group_by(match_id, round_num, map) %>%
  slice_max(tick, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  select(match_id, round_num, map, seconds_since_plant)

planted_bomb_outcomes <- plant_timing %>%
  inner_join(
    post_plant_timing,
    by = c("match_id", "round_num", "map")
  ) %>%
  mutate(
    outcome = case_when(
      reason_name == "bomb_defused" ~ "Bomb defused",
      reason_name == "target_bombed" ~ "Bomb exploded",
      is.na(reason_name) ~ "Unknown outcome",
      TRUE ~ as.character(reason_name)
    )
  ) %>%
  group_by(map, outcome) %>%
  summarise(
    rounds = n(),

    mean_seconds_until_plant = mean(seconds, na.rm = TRUE),
    median_seconds_until_plant = median(seconds, na.rm = TRUE),
    sd_seconds_until_plant = sd(seconds, na.rm = TRUE),

    mean_seconds_after_plant =
      mean(seconds_since_plant, na.rm = TRUE),
    median_seconds_after_plant =
      median(seconds_since_plant, na.rm = TRUE),
    sd_seconds_after_plant =
      sd(seconds_since_plant, na.rm = TRUE),

    .groups = "drop"
  ) %>%
  collect()

planted_bomb_outcomes
```

```
# A tibble: 7 x 9
  map       outcome         rounds mean_seconds_until_p~1 median_seconds_until~2
  <chr>     <chr>            <int>                  <dbl>                   <dbl>
1 de_dust2  Bomb defused        79                   82.2                      89
2 de_dust2  Bomb exploded      129                   77.0                      79
3 de_dust2  terrorists_win     124                   78.5                      82
4 de_mirage Bomb defused        75                   71.8                      79
5 de_mirage Bomb exploded      105                   75.1                      80
6 de_mirage Unknown outcome      7                   84.1                      91
7 de_mirage terrorists_win     107                   74.2                      76
# i abbreviated names: 1: mean_seconds_until_plant,
#   2: median_seconds_until_plant
# i 4 more variables: sd_seconds_until_plant <dbl>,
#   mean_seconds_after_plant <dbl>, median_seconds_after_plant <dbl>,
#   sd_seconds_after_plant <dbl>
```

```r
# Median times until and after the plant, in long format for faceting.
plot_data <- planted_bomb_outcomes %>%
  select(
    map, outcome,
    until_plant = median_seconds_until_plant,
    after_plant = median_seconds_after_plant
  ) %>%
  pivot_longer(
    c(until_plant, after_plant),
    names_to = "timing",
    values_to = "median_seconds"
  ) %>%
  mutate(
    timing = recode(
      timing,
      until_plant = "Until plant",
      after_plant = "After plant"
    )
  )

ggplot(
  plot_data,
  aes(x = outcome, y = median_seconds, fill = outcome)
) +
  geom_col(show.legend = FALSE) +
  geom_text(
```

```
  aes(label = sprintf("%.1f", median_seconds)),
  vjust = -0.4,
  size = 3
) +
facet_grid(timing ~ map, scales = "free_y") +
scale_y_continuous(
  expand = expansion(mult = c(0, 0.15))
) +
labs(
  title = "Bomb Plant Timing by Map and Round Outcome",
  x = "Round outcome",
  y = "Median time (seconds)"
) +
theme_minimal() +
theme(
  axis.text.x = element_text(angle = 35, hjust = 1),
  strip.text = element_text(face = "bold"),
  panel.spacing = unit(1, "lines")
)
```

